

Contents

Toolkit-Referenz	1
Distanz & Ausrichtung	1
Arena-Geometrie	2
Gegner-Suche	3
Kampf & Sensoren-Wrapper	3
Was das Toolkit bewusst NICHT macht	5

Toolkit-Referenz

Funktionsübersicht für alle Helfer aus `framework.arena (Toolkit.kt)`. Diese Funktionen nehmen euch die Raster-Rechnerei ab (Distanz, Richtung, Ausrichtung, Rand/Mitte, Gegnersuche) — ihr müsst sie nicht selbst herleiten, nur aufrufen.

Ein `import framework.arena.*` reicht, um alle Funktionen nutzen zu können — kein zusätzlicher Import-Pfad nötig.

Kein echtes Team-Konzept: `named/enemiesNamed` filtern nach dem tatsächlichen Bot-Namen (`RobotBrain.name`), nicht nach einer echten Team-Zugehörigkeit zur Laufzeit. Team A/B/C ist nur eine Organisation für die Bot-Auswahl in der UI — im Kampf ist jeder Bot für sich (siehe `CLAUDE.md`).

Distanz & Ausrichtung

Funktionen auf `Position`.

Funktion	Signatur	Was sie tut
<code>manhattanDistanceTo</code>	<code>Position.manhattanDistanceTo(other: Position): Int</code>	Anzahl Felder bis zum Ziel, wenn man nur waagrecht/senkrecht laufen darf (keine Diagonale). Beispiel: 3 Felder nach rechts und 2 nach unten = Abstand 5.
<code>isAlignedWith</code>	<code>Position.isAlignedWith(other: Position): Boolean</code>	<code>true</code> , wenn <code>other</code> in derselben Spalte oder derselben Reihe steht — also ob ein Schuss dorthin überhaupt treffen könnte.
<code>directionTo</code>	<code>Position.directionTo(other: Position): Direction?</code>	Richtung zu <code>other</code> , aber nur wenn diese gerade (nicht diagonal) erreichbar ist. Sonst <code>null</code> — also vorher mit <code>isAlignedWith</code> oder <code>canShoot</code> checken.

Funktion	Signatur	Was sie tut
approachDirectionTo	Position.approachDirectionTo(Position): Direction?	Ein Schritt Richtung other — funktioniert auch diagonal, macht dann einfach den größeren der beiden Schritte zuerst. Jeder Aufruf bringt euch garantiert ein Feld näher. null nur, wenn ihr schon auf demselben Feld steht.
fleeDirectionFrom	Position.fleeDirectionFrom(Position): Direction?	Ein Schritt weg von other — das Gegenteil von approachDirectionTo. Achtung: läuft stur weg, ohne auf die Arena-Wand zu achten (siehe unten).

Beispiel:

```

val ziel = sensors.nearestEnemy() ?: return Action.Wait
val self = sensors.self.position

if (self.isAlignedWith(ziel.position)) {
    return Action.Shoot(self.directionTo(ziel.position)!!)
}
return Action.Move(self.approachDirectionTo(ziel.position)!!)

```

Arena-Geometrie

Funktionen auf Position (brauchen arenaWidth/arenaHeight).

Funktion	Signatur	Was sie tut
isInsideArena	Position.isInsideArena(arenaWidth: Int, arenaHeight: Int): Boolean	Wird, wenn die Position noch innerhalb des Feldes liegt. Praktisch, um nach approachDirectionTo/fleeDirectionFrom zu checken, ob der Zug nicht gegen die Wand geht.
distanceToNearestEdge	Position.distanceToNearestEdge(arenaWidth: Int, arenaHeight: Int): Int	Wie viele Felder bis zum nächstgelegenen Rand. In der Ecke: 0. In der Mitte einer 10×10-Arena: 4.
directionToNearestEdge	Position.directionToNearestEdge(arenaWidth: Int, arenaHeight: Int): Direction	Richtungsangabe zum nächstgelegenen Rand.

Funktion	Signatur	Was sie tut
isNearCenter	Position.isNearCenter(arenaWidth: Int, arenaHeight: Int, margin: Int = 2): Boolean	Wird, wenn die Position nah am Mittelpunkt der Arena ist (Standard: ± 2 Felder in beide Richtungen). Nützlich, um das gefährliche Zentrum zu meiden (Story 1.2).

Beispiel:

```
if (sensors.isNearCenter()) {
    return Action.Move(sensors.directionToNearestEdge())
}
```

Gegner-Suche

Funktionen auf `List<RobotState>` (wiederverwendbar, z.B. auf `sensors.others` oder gefilterten Zwischenergebnissen).

Funktion	Signatur	Was sie tut
nearestTo	<code>List<RobotState>.nearestTo(Position): RobotState?</code>	Roboter mit dem kleinsten Abstand zu <code>position</code> . null, wenn die Liste leer ist.
weakest	<code>List<RobotState>.weakest(): RobotState?</code>	Roboter mit den wenigsten HP. null, wenn die Liste leer ist.
strongest	<code>List<RobotState>.strongest(): RobotState?</code>	Roboter mit den meisten HP. null, wenn die Liste leer ist.
inLineWith	<code>List<RobotState>.inLineWith(Position): List<RobotState></code>	Alle Roboter aus der Liste, die von <code>position</code> aus gerade (nicht diagonal) erreichbar sind — also mögliche Schussziele. Kann eine leere Liste sein.
named	<code>List<RobotState>.named(name: String): List<RobotState></code>	Alle Roboter mit exakt diesem Namen (<code>RobotBrain.name</code>). Kein echtes Team-Konzept: zwei Bots mit gleichem Namen zählen hier als "gleich", egal aus welchem <code>teamXBots</code> sie kommen.

Praktischer sind meist die Wrapper direkt auf `Sensors` (siehe unten) — die nehmen euch `sensors.others/sensors.self.position` schon ab.

Kampf & Sensoren-Wrapper

Funktionen direkt auf `Sensors` — kombinieren die obigen Bausteine mit `sensors.others/sensors.self`, damit ihr nicht jedes Mal `sensors.others` und `sensors.self.position` selbst durchreichen müsst.

Funktion	Signatur	Was sie tut
nearestEnemy	Sensors.nearestEnemy(): RobotState?	Nächster lebender Gegner. null, wenn keiner mehr lebt.
weakestEnemy	Sensors.weakestEnemy(): RobotState?	Gegner mit den wenigsten HP.
strongestEnemy	Sensors.strongestEnemy(): RobotState?	Gegner mit den meisten HP.
enemiesInLineOfSight	Sensors.enemiesInLineOfSight(): List<RobotState>	Alle lebenden Gegner, die gerade (nicht diagonal) getroffen werden könnten.
nearestEnemyInLineOfSight	Sensors.nearestEnemyInLineOfSight(): RobotState?	Nächster Gegner in Sichtlinie. null, wenn gerade keiner in Reihe/Spalte steht — auch wenn insgesamt noch Gegner leben.
canShoot	Sensors.canShoot(target: RobotState): Boolean	true, wenn target gerade (nicht diagonal) zu treffen wäre. Prüft nicht, ob target noch lebt oder ob ein anderer Bot dazwischensteht.
isNearCenter	Sensors.isNearCenter(margin: Int = 2): Boolean	Steht der eigene Bot nah am Zentrum?
distanceToNearestEdge	Sensors.distanceToNearestEdge(): Int	Eigener Abstand zum nächsten Rand.
directionToNearestEdge	Sensors.directionToNearestEdge(): Direction	Richtung zum nächsten Rand von der eigenen Position aus.
enemiesNamed	Sensors.enemiesNamed(name: String): List<RobotState>	Lebende Gegner mit passendem Namen (Team-Hinweis oben gilt auch hier).

Beispiel — kompletter Angriff-mit-Flucht-Bot:

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val ziel = sensors.nearestEnemy() ?: return Action.Wait
        val self = sensors.self.position

        if (sensors.self.health < 20) {
            return Action.Move(self.fleeDirectionFrom(ziel.position)!!)
        }
        if (sensors.canShoot(ziel)) {
            return Action.Shoot(self.directionTo(ziel.position)!!)
        }
        return Action.Move(self.approachDirectionTo(ziel.position)!!)
    }
}

```

Was das Toolkit bewusst NICHT macht

- Keine Arena-Randbeachtung in `fleeDirectionFrom/approachDirectionTo` — reine Achsen-Heuristik. Wer das kombinieren will (z.B. “fliehe, aber nicht gegen die Wand”), kann das selbst mit `isInsideArena` bauen (gute Kür-Aufgabe, Story 3.3).
- Kein Gedächtnis über mehrere Ticks (z.B. “wohin ist der Gegner zuletzt gelaufen”) — jede Funktion arbeitet nur mit der aktuellen Sensors-Momentaufnahme.
- Kein echtes Team-/Friendly-Fire-Konzept, siehe Hinweis oben zu `named`.